



**Bikkad IT Institute**

---

# **Pandas Series Notes**

*Compiled by Bikkad IT Institute | Teaching Notes*

---

---

## **OUR LOCATIONS**

**+91 77220 82225**  
**Pimpri-Chinchwad**

**+91 77220 82226**  
**Chh. Sambhajinagar**

**+91 77220 82227**  
**Nashik**

**+91 77220 82224**  
**Hadapsar, Pune**

# Pandas Series Notes

## Creating, Exploring, Indexing and Cleaning a Series - Step by Step

Prepared for Bikkad IT Institute Students | Level: Beginner to Intermediate | Environment: Jupyter Notebook or Google Colab

### Topic Index

1. Creating a Series
2. Series Attributes
3. Creating a Series from a CSV File
4. Viewing a Series (head, tail, sample)
5. Sorting a Series
6. Counting Values (value\_counts & nunique)
7. Series Statistical / Math Methods
8. Indexing and Slicing a Series
9. Editing a Series
10. Series with Python Built-ins
11. Arithmetic and Relational Operators
12. Boolean Indexing on a Series
13. Handling Missing Values
14. astype, between and clip
15. drop\_duplicates, isin and apply
16. Plotting a Series

**Datasets used throughout:** canteen\_sales.csv (daily samosa sales at a college canteen), cricket\_star\_runs.csv (a rising IPL batter's innings-by-innings runs), and superhero\_movies.csv (superhero films and their lead actors). Run the examples in order, like cells in one Jupyter notebook: pandas is imported once in Topic 1, the three datasets are read once in Topic 3, and every later example reuses them.

## 1. Creating a Series

A Pandas Series is a one-dimensional labeled array - like a single column in a spreadsheet. It can be built from a plain Python list or a dictionary, and can carry a custom index and a name.

### Syntax

```
pd.Series(data, index=None, name=None)
```

#### OUR LOCATIONS

+91 77220 82225  
Pimpri-Chinchwad

+91 77220 82226  
Chh. Sambhajinagar

+91 77220 82227  
Nashik

+91 77220 82224  
Hadapsar, Pune

**Example 1: Series from a list, with a custom index and name**

```
import pandas as pd

scores = [78, 92, 65, 88]
print(scores)

students = ['Aakash', 'Priya', 'Rahul', 'Sneha']
print(students)

quiz = pd.Series(scores)
print(quiz)

quiz = pd.Series(scores, index=students)
print(quiz)

quiz = pd.Series(scores, index=students, name='Quiz 1 Marks')
print(quiz)
```

**Output / Result**

```
[78, 92, 65, 88]
['Aakash', 'Priya', 'Rahul', 'Sneha']
0      78
1      92
2      65
3      88
dtype: int64
Aakash    78
Priya     92
Rahul     65
Sneha     88
dtype: int64
Aakash    78
Priya     92
Rahul     65
Sneha     88
Name: Quiz 1 Marks, dtype: int64
```

**Explanation**

The first two lines print the two plain Python lists. The first Series uses the default index 0, 1, 2, 3. The second Series replaces those numbers with the student names - the two lists are matched by position. The third Series is the same but with a name, which appears in the last line of the output together with dtype: int64 (whole numbers).

**OUR LOCATIONS**

### Example 2: Series from a dictionary

```
medals = {'India': 7, 'USA': 126, 'China': 88, 'Japan': 45}
print(medals)

print(list(medals.keys()))
print(list(medals.values()))

olympic = pd.Series(medals, name='Olympic Medals 2024')
print(olympic)
```

#### Output / Result

```
{'India': 7, 'USA': 126, 'China': 88, 'Japan': 45}
['India', 'USA', 'China', 'Japan']
[7, 126, 88, 45]
India      7
USA       126
China     88
Japan     45
Name: Olympic Medals 2024, dtype: int64
```

#### Explanation

The first line is the dictionary, the next two lines are its keys and its values. In the Series, the keys became the index and the values became the data - a dictionary needs no separate index= argument.

## 2. Series Attributes

Every Series carries metadata describing its size, storage type, and labels - useful for quickly understanding data you did not create yourself.

#### Syntax

```
s.size; s.dtype; s.name; s.index; s.values; s.is_unique
```

#### OUR LOCATIONS

+91 77220 82225  
Pimpri-Chinchwad

+91 77220 82226  
Chh. Sambhajinagar

+91 77220 82227  
Nashik

+91 77220 82224  
Hadapsar, Pune

### Example 1: size, dtype, name and index

```
print(quiz)

size = quiz.size
print(size)

data_type = quiz.dtype
print(data_type)

series_name = quiz.name
print(series_name)

labels = quiz.index
print(labels)

label_list = quiz.index.tolist()
print(label_list)
```

#### Output / Result

```
Aakash    78
Priya     92
Rahul     65
Sneha     88
Name: Quiz 1 Marks, dtype: int64
4
int64
Quiz 1 Marks
Index(['Aakash', 'Priya', 'Rahul', 'Sneha'], dtype='str')
['Aakash', 'Priya', 'Rahul', 'Sneha']
```

#### Explanation

After the Series itself, the output shows: 4 (size - how many values), int64 (dtype - whole numbers), Quiz 1 Marks (the name), the Index object, and finally the same index as a plain Python list using .tolist(), which is easier to read.

#### OUR LOCATIONS

### Example 2: values and is\_unique

```
print(olympic)

only_values = olympic.values
print(only_values)
print(type(only_values))

print(olympic.is_unique)

repeat = pd.Series([1, 1, 2, 3])
print(repeat.tolist())
print(repeat.is_unique)
```

#### Output / Result

```
India      7
USA       126
China      88
Japan      45
Name: Olympic Medals 2024, dtype: int64
[ 7 126  88  45]
<class 'numpy.ndarray'>
True
[1, 1, 2, 3]
False
```

#### Explanation

.values gives only the data without the index, and the next line confirms it is a NumPy array. True means every medal count is different. [1, 1, 2, 3] has the value 1 twice, so its is\_unique is False.

## 3. Creating a Series from a CSV File

read\_csv() normally returns a two-dimensional DataFrame; chaining .squeeze('columns') collapses a single-column result down into a one-dimensional Series.

#### Syntax

```
pd.read_csv(path).squeeze('columns')
pd.read_csv(path, index_col='col').squeeze('columns')
```

#### OUR LOCATIONS

**Example 1: A single-column CSV (default RangeIndex)**

```
df = pd.read_csv('datasets/canteen_sales.csv')
print(type(df))
print(df.shape)

canteen = df.squeeze('columns')
print(type(canteen))

print(canteen.head())
```

**Output / Result**

```
<class 'pandas.DataFrame'>
(200, 1)
<class 'pandas.Series'>
0    64
1    59
2    65
3    72
4    58
Name: Samosas_Sold, dtype: int64
```

**Explanation**

`read_csv()` always returns a DataFrame - here with shape (200, 1): 200 rows and 1 column. After `squeeze('columns')` the type becomes Series. The column name `Samosas_Sold` becomes the Series name, and the index is the default 0, 1, 2, ...

**Example 2: A two-column CSV with a custom index**

```
df = pd.read_csv('datasets/cricket_star_runs.csv')
print(df.head(3))

df = pd.read_csv('datasets/cricket_star_runs.csv', index_col='match_no')
print(df.head(3))

cricket = df.squeeze('columns')
print(type(cricket))
print(cricket.head())

heroes = pd.read_csv('datasets/superhero_movies.csv',
                     index_col='movie').squeeze('columns')
print(heroes.head())
```

**OUR LOCATIONS**

Output / Result

```
match_no  runs
0         1   85
1         2   65
2         3   60
runs
match_no
1         85
2         65
3         60
<class 'pandas.Series'>
match_no
1         85
2         65
3         60
4         60
5         25
Name: runs, dtype: int64
movie
Iron Man           Robert Downey Jr.
Iron Man 2         Robert Downey Jr.
Iron Man 3         Robert Downey Jr.
The Avengers       Robert Downey Jr.
Avengers: Age of Ultron  Robert Downey Jr.
Name: lead_actor, dtype: str
```

Explanation

The first table shows the file has two columns, match\_no and runs. With index\_col='match\_no', match\_no moves into the index and only runs is left as a column. Now squeeze('columns') can turn it into a Series whose labels are the match numbers 1, 2, 3... The superhero file is loaded the same way, with the movie titles as the index. canteen, cricket and heroes are now ready - every later example reuses them, so the files never need to be read again.

Teaching Note

Older pandas code often used pd.read\_csv(path, squeeze=True). That parameter was removed in modern pandas - .squeeze('columns') is the current, supported way to get the same one-column-to-Series conversion.

4. Viewing a Series (head, tail, sample)

For a large Series, printing everything is impractical; head, tail, and sample preview a manageable slice of the data.

Syntax

```
s.head(n=5); s.tail(n=5); s.sample(n, random_state=None)
```

### Example 1: head and tail

```
print(canteen.size)

first_five = canteen.head()
print(first_five)

last_three = canteen.tail(3)
print(last_three)
```

#### Output / Result

```
200
0      64
1      59
2      65
3      72
4      58
Name: Samosas_Sold, dtype: int64
197      76
198      75
199      66
Name: Samosas_Sold, dtype: int64
```

#### Explanation

200 is the number of days in the file. head() with no number shows the first 5 values (index 0 to 4). tail(3) shows the last 3 values - index 197, 198 and 199 are the final days.

### Example 2: A random sample of movies

```
print(heroes.size)

random_five = heroes.sample(5, random_state=7)
print(random_five)

same_five = heroes.sample(5, random_state=7)
print(random_five.equals(same_five))
```

#### OUR LOCATIONS

Output / Result

```
95
movie
Spider-Man: No Way Home      Tom Holland
Krrish 3                     Hrithik Roshan
Hellboy                      Ron Perlman
Wonder Woman 1984           Gal Gadot
Deadpool                    Ryan Reynolds
Name: lead_actor, dtype: str
True
```

Explanation

95 is the number of movies. sample(5) picks 5 rows at random. random\_state=7 fixes the randomness, so picking again with the same number gives the same 5 movies - that is why the last line is True. Without random\_state you get different movies every time.

5. Sorting a Series

A Series can be sorted by its values or by its index, in either direction, and either returned as a new Series or applied in place.

Syntax

```
s.sort_values(ascending=True, inplace=False)
s.sort_index(ascending=True, inplace=False)
```

Example 1: Sorting values - the batter's best innings

```
print(cricket.head())

sorted_runs = cricket.sort_values(ascending=False)
print(sorted_runs.head())
```

### Output / Result

```
match_no
1      85
2      65
3      60
4      60
5      25
Name: runs, dtype: int64
match_no
88      128
127     110
103     100
42      100
50      100
Name: runs, dtype: int64
```

### Explanation

The first output is in match order. After `sort_values(ascending=False)` the runs go from highest to lowest. Each score keeps its match number in the index, so we can see the best innings, 128 runs, came in match 88.

### Example 2: Sorting the index alphabetically

```
print(heroes.head(3))

by_title = heroes.sort_index()
print(by_title.head())
```

### Output / Result

```
movie
Iron Man      Robert Downey Jr.
Iron Man 2    Robert Downey Jr.
Iron Man 3    Robert Downey Jr.
Name: lead_actor, dtype: str
movie
A Flying Jatt      Tiger Shroff
Ant-Man            Paul Rudd
Ant-Man and the Wasp      Paul Rudd
Ant-Man and the Wasp: Quantumania  Paul Rudd
Aquaman           Jason Momoa
Name: lead_actor, dtype: str
```

### Explanation

The first output shows the movies in file order. `sort_index()` sorts by the labels (the movie titles), not the values, so titles starting with 'A' come first. Both sort methods return a new Series; add `inplace=True` to change the original instead.

#### OUR LOCATIONS

## 6. Counting Values (value\_counts & nunique)

value\_counts() tallies how often each distinct value appears - one of the most useful methods for exploring categorical data such as names or labels.

### Syntax

```
s.value_counts(); s.nunique()
```

### Example 1: Which actor leads the most superhero films?

```
print(heroes.head(3))

actor_counts = heroes.value_counts()
print(type(actor_counts))

print(actor_counts.head())
```

Output / Result

movie  
Iron Man            Robert Downey Jr.  
Iron Man 2        Robert Downey Jr.  
Iron Man 3        Robert Downey Jr.  
Name: lead\_actor, dtype: str  
<class 'pandas.Series'>  
lead\_actor  
Robert Downey Jr.        7  
Hugh Jackman            6  
Chris Hemsworth        5  
Chris Evans            4  
Tom Holland            4  
Name: count, dtype: int64

### Explanation

value\_counts() returns a new Series - the type line confirms it. The actor names become the index and the number of films becomes the data, sorted from most to least. Robert Downey Jr. leads with 7 films.

### Example 2: Counting distinct actors overall

```
distinct_actors = heroes.nunique()
print(distinct_actors)

total_movies = heroes.size
print(total_movies)

print(round(total_movies / distinct_actors, 1))
```

#### Output / Result

```
45
95
2.1
```

#### Explanation

45 is nunique() - how many different actors there are. 95 is size - every row (movie).  $95 / 45 = 2.1$ , so each actor leads about 2 films on average.

## 7. Series Statistical / Math Methods

A numeric Series supports the full set of descriptive statistics out of the box, from simple totals to a full summary in one call.

#### Syntax

```
s.sum(); s.mean(); s.median(); s.std(); s.var()
s.min(); s.max(); s.describe()
```

### Example 1: Totals and central tendency

```
total = canteen.sum()
print(total)

average = canteen.mean()
print(average)
print(round(average, 1))

middle = canteen.median()
print(middle)
```

#### OUR LOCATIONS

Output / Result

14189  
70.945  
70.9  
70.0

Explanation

14189 is the total from sum(). 70.945 is mean() = 14189 / 200 days, and 70.9 is the same value rounded to 1 decimal. 70.0 is the median() - the middle value after sorting. Mean and median are close, so a typical day sells about 70 samosas.

Example 2: Spread and a full summary

```
spread = cricket.std()
print(round(spread, 1))

best = cricket.max()
print(best)

worst = cricket.min()
print(worst)

summary = cricket.describe()
print(summary)
```

Output / Result

30.5  
128  
0  
count 130.000000  
mean 42.169231  
std 30.543688  
min 0.000000  
25% 18.000000  
50% 40.000000  
75% 65.000000  
max 128.000000  
Name: runs, dtype: float64

Explanation

30.5 is std() - scores usually differ from the average by about 30 runs, so the batter is not very consistent. 128 is the best innings (max) and 0 the worst (min). describe() puts count, mean, std, min, the 25% / 50% / 75% quartiles and max into one table.

## 8. Indexing and Slicing a Series

Individual values can be reached by their integer position with `.iloc`, by their label with `.loc`, or with the classic square-bracket shortcut for both.

### Syntax

```
s.iloc[position]; s.loc[label]; s[start:stop]
```

### Example 1: Positional vs label access

```
print(cricket.head(3))

by_position = cricket.iloc[0]
print(by_position)

by_label = cricket.loc[1]
print(by_label)

by_name = heroes['Iron Man']
print(by_name)
```

Output / Result

match\_no  
1      85  
2      65  
3      60  
Name: runs, dtype: int64  
85  
85  
Robert Downey Jr.

### Explanation

The first 85 comes from `iloc[0]` - 'the first row', whatever its label. The second 85 comes from `loc[1]` - 'the row whose label is 1'. Both are the same here because match 1 is also the first row. For a text index, `heroes['Iron Man']` finds the row by its label.

### Example 2: Slicing and fancy (list) indexing

```
five_innings = cricket[5:10]
print(five_innings)

wanted = [1, 2, 3]
print(wanted)

picked = cricket.loc[wanted]
print(picked)
```

**Output / Result**

```
match_no
6      22
7      75
8      75
9      80
10     90
Name: runs, dtype: int64
[1, 2, 3]
match_no
1      85
2      65
3      60
Name: runs, dtype: int64
```

**Explanation**  
A slice with [ ] is positional: [5:10] starts at position 5 (the 6th row) and stops before position 10, so we get 5 rows - matches 6 to 10. Passing the list [1, 2, 3] to .loc picks exactly those labels - matches 1, 2 and 3.

## 9. Editing a Series

Values can be changed in place through the same indexing styles used to read them, and assigning to a brand-new label extends the Series with a new row.

**Syntax**

```
s.loc[label] = value; s.iloc[pos] = value
s[new_label] = value      # adds a new row
```

**Example 1: Updating a value and adding a new row**

```
print(quiz)

quiz.loc['Aakash'] = 85
print(quiz)

quiz['Zoya'] = 91
print(quiz)
```

**Output / Result**

```
Aakash    78
Priya     92
Rahul     65
Sneha     88
Name: Quiz 1 Marks, dtype: int64
Aakash    85
Priya     92
Rahul     65
Sneha     88
Name: Quiz 1 Marks, dtype: int64
Aakash    85
Priya     92
Rahul     65
Sneha     88
Zoya      91
Name: Quiz 1 Marks, dtype: int64
```

**Explanation**

The three outputs show the Series before editing, after `quiz.loc['Aakash'] = 85` (78 becomes 85), and after `quiz['Zoya'] = 91`. Zoya did not exist yet, so pandas adds her as a new row at the end.

**Example 2: Slice and fancy-index assignment**

```
runs_copy = cricket.copy()
print(runs_copy.head(6))

runs_copy.iloc[0:3] = [150, 150, 150]
print(runs_copy.head(6))

runs_copy.loc[[4, 5]] = 0
print(runs_copy.head(6))

print(cricket.head(3).tolist())
```

**OUR LOCATIONS**

Output / Result

```
match_no
1      85
2      65
3      60
4      60
5      25
6      22
Name: runs, dtype: int64
match_no
1      150
2      150
3      150
4       60
5       25
6       22
Name: runs, dtype: int64
match_no
1      150
2      150
3      150
4        0
5        0
6       22
Name: runs, dtype: int64
[85, 65, 60]
```

Explanation

The second output shows `iloc[0:3] = [150, 150, 150]` changing the first three positions. The third shows `loc[[4, 5]] = 0` changing matches 4 and 5 by label. The last line `[85, 65, 60]` proves the original cricket Series did not change, because we edited a copy.

Teaching Note

`runs_copy = cricket.copy()` was used deliberately: without `.copy()`, some Series operations return a view that shares memory with the original, so editing the copy could silently change cricket too.

10. Series with Python Built-ins

A Series cooperates with Python's own built-in functions and the membership operator, since it behaves much like a list-plus-dictionary hybrid.

Syntax

```
len(s); sorted(s); min(s); max(s)
label in s.index; value in s.values
```

### Example 1: len, sorted, min and max

```
days = len(canteen)
print(days)

lowest = min(canteen)
print(lowest)

highest = max(canteen)
print(highest)

sorted_list = sorted(canteen)
print(type(sorted_list))
print(sorted_list[:5])
```

#### Output / Result

```
200
30
142
<class 'list'>
[30, 36, 39, 41, 45]
```

#### Explanation

Python's own len(), min() and max() work on a Series like on a list: 200 days, lowest 30, highest 142. sorted() also works, but the next line shows it returns a plain list, so we slice it with [:5] to see the 5 smallest sales.

### Example 2: Type conversion and membership

```
as_list = list(quiz)
print(as_list[:3])

as_dict = dict(quiz.head(3))
print(as_dict)

in_index = 'Iron Man' in heroes.index
print(in_index)

in_values = 'Robert Downey Jr.' in heroes.values
print(in_values)
```

#### Output / Result

```
[85, 92, 65]
{'Aakash': np.int64(85), 'Priya': np.int64(92), 'Rahul': np.int64(65)}
True
True
```

#### OUR LOCATIONS

Explanation

quiz is the Series we edited in Topic 9, so Aakash now has 85. list() keeps only the values; dict() keeps label: value pairs. np.int64(85) is simply the number 85 stored as a NumPy integer. The first True says 'Iron Man' is in the index; the second True says 'Robert Downey Jr.' is in the values. To search for an actor's name we must check heroes.values.

# 11. Arithmetic and Relational Operators

Arithmetic and comparison operators broadcast across a Series automatically, applying the same operation to every value without an explicit loop.

Syntax

```
s + value; s * value
s >= value      # returns a Boolean Series
```

## Example 1: Broadcasting - 5 extra samosas every day

```
print(canteen.head())

with_offer = canteen + 5
print(with_offer.head())
```

Output / Result

|   |    |
|---|----|
| 0 | 64 |
| 1 | 59 |
| 2 | 65 |
| 3 | 72 |
| 4 | 58 |

Name: Samosas\_Sold, dtype: int64

|   |    |
|---|----|
| 0 | 69 |
| 1 | 64 |
| 2 | 70 |
| 3 | 77 |
| 4 | 63 |

Name: Samosas\_Sold, dtype: int64

Explanation

canteen + 5 adds 5 to every value without a loop - 64 becomes 69, 59 becomes 64 and so on. This is called broadcasting. The result is a new Series; canteen itself is not changed.

### Example 2: A relational operator returns True/False per row

```
print(cricket.head())

fifty_plus = cricket.ge(50)
print(fifty_plus.head())

same_thing = cricket >= 50
print(fifty_plus.equals(same_thing))
```

#### Output / Result

```
match_no
1      85
2      65
3      60
4      60
5      25
Name: runs, dtype: int64
match_no
1      True
2      True
3      True
4      True
5     False
Name: runs, dtype: bool
True
```

#### Explanation

`cricket.ge(50)` compares every score with 50 and returns True or False for each innings - match 5 scored 25, so it is False. `ge` means 'greater than or equal to'; the last line `True` confirms it gives exactly the same result as `>=`.

## 12. Boolean Indexing on a Series

Wrapping a condition in square brackets filters a Series down to only the rows where that condition is True.

#### Syntax

```
s[condition]
```

#### OUR LOCATIONS

+91 77220 82225  
Pimpri-Chinchwad

+91 77220 82226  
Chh. Sambhajinagar

+91 77220 82227  
Nashik

+91 77220 82224  
Hadapsar, Pune

### Example 1: Counting half-centuries and ducks

```
mask_50 = cricket >= 50
print(mask_50.head().tolist())

fifties = cricket[mask_50]
print(fifties.head())
print(fifties.size)

mask_duck = cricket == 0
ducks = cricket[mask_duck]
print(ducks.size)
```

#### Output / Result

```
[True, True, True, True, False]
match_no
1      85
2      65
3      60
4      60
7      75
Name: runs, dtype: int64
57
21
```

#### Explanation

The first line is the condition for the first 5 innings. Putting the condition inside `cricket[ ]` keeps only the rows where it is True - notice match 5 and 6 are skipped. `.size` counts the rows that survived: 57 innings of 50 or more, and 21 ducks (0 runs).

### Example 2: Actors who lead more than 3 superhero films

```
top_billed = heroes.value_counts()
print(top_billed.head(7))

more_than_3 = top_billed > 3
print(more_than_3.sum())

print(top_billed[more_than_3])
```

#### OUR LOCATIONS

### Output / Result

```
lead_actor
Robert Downey Jr.    7
Hugh Jackman         6
Chris Hemsworth      5
Chris Evans          4
Tom Holland          4
Tobey Maguire        3
Chris Pratt          3
Name: count, dtype: int64
5
lead_actor
Robert Downey Jr.    7
Hugh Jackman         6
Chris Hemsworth      5
Chris Evans          4
Tom Holland          4
Name: count, dtype: int64
```

### Explanation

value\_counts() gives films per actor. top\_billed > 3 marks actors with more than 3 films as True, and sum() counts the Trues - 5. Filtering with top\_billed[more\_than\_3] keeps only those 5 actors.

## 13. Handling Missing Values

Real datasets often have gaps. Pandas marks a missing numeric value as NaN, with dedicated methods to detect, drop, or fill those gaps.

### Syntax

```
s.isnull(); s.dropna(); s.fillna(value)
```

### Example 1: Detecting missing values

```
import numpy as np

temp = pd.Series([72, 75, np.nan, 69, np.nan, 80])
print(temp)

missing_mask = temp.isnull()
print(missing_mask)

missing_count = missing_mask.sum()
print(missing_count)
```

**Output / Result**

```
0    72.0
1    75.0
2     NaN
3    69.0
4     NaN
5    80.0
dtype: float64
0    False
1    False
2     True
3    False
4     True
5    False
dtype: bool
2
```

**Explanation**

np.nan means 'missing'. Because of it, the whole Series becomes float64 (72.0 instead of 72). isnull() is True at positions 2 and 4, and sum() counts each True as 1, giving 2 missing values.

**Example 2: Dropping vs filling the gaps**

```
dropped = temp.dropna()
print(dropped)

average = temp.mean()
print(average)

filled = temp.fillna(average)
print(filled)
```

**Output / Result**

```
0    72.0
1    75.0
3    69.0
5    80.0
dtype: float64
74.0
0    72.0
1    75.0
2    74.0
3    69.0
4    74.0
5    80.0
dtype: float64
```

**OUR LOCATIONS**

**Explanation**

dropna() removes rows 2 and 4 completely, leaving 4 values. 74.0 is mean(), which ignores NaN:  $(72 + 75 + 69 + 80) / 4 = 74.0$ . fillna(average) puts 74.0 in both gaps and keeps all 6 rows.

14. astype, between and clip

astype converts a Series to a more suitable data type, between checks whether values fall in a range, and clip caps values to a minimum and maximum.

**Syntax**

```
s.astype(dtype); s.between(left, right); s.clip(lower, upper)
```

Example 1: Shrinking memory with astype

```
import sys

print(cricket.dtype)
print(sys.getsizeof(cricket))

small = cricket.astype('int16')
print(small.dtype)
print(sys.getsizeof(small))

print((small == cricket).all())
```

**Output / Result**

int64  
1204  
int16  
424  
True

**Explanation**

The Series starts as int64 using 1204 bytes. After astype('int16') it uses 424 bytes, because int16 needs 2 bytes per number instead of 8. Runs never go above 32,767 (the int16 limit), so the last line True confirms every value stayed the same.

### Example 2: between and clip

```
in_range = cricket.between(50, 99)
print(in_range.head().tolist())
print(in_range.sum())

print(canteen.min())
print(canteen.max())

capped = canteen.clip(50, 100)
print(capped.min())
print(capped.max())
print(capped.head())
```

#### Output / Result

```
[True, True, True, True, False]
52
30
142
50
100
0    64
1    59
2    65
3    72
4    58
Name: Samosas_Sold, dtype: int64
```

#### Explanation

`between(50, 99)` is True when a score is from 50 to 99, both ends included - 52 such innings. Canteen sales go from 30 to 142 before clip and from 50 to 100 after `clip(50, 100)`: values below 50 become 50 and values above 100 become 100. Values already inside the range, like 64, stay the same.

## 15. drop\_duplicates, isin and apply

`drop_duplicates` removes repeated values, `isin` tests membership against a list of candidates, and `apply` runs a custom function over every value.

#### Syntax

```
s.drop_duplicates(); s.isin(values); s.apply(function)
```

#### OUR LOCATIONS

Example 1: Unique lead actors and a target-score filter

```
print(heroes.size)

first_movie_per_actor = heroes.drop_duplicates()
print(first_movie_per_actor.size)
print(first_movie_per_actor.head(3))

targets = [0, 100, 110, 120, 128]
print(targets)

mask = cricket.isin(targets)
print(mask.sum())

matched = cricket[mask]
print(matched.value_counts())
```

**Output / Result**

```
95
45
movie
Iron Man                                Robert Downey Jr.
Captain America: The First Avenger      Chris Evans
Thor                                    Chris Hemsworth
Name: lead_actor, dtype: str
[0, 100, 110, 120, 128]
26
runs
0      21
100     3
128     1
110     1
Name: count, dtype: int64
```

**Explanation**

95 rows become 45 after drop\_duplicates(), which keeps only the first movie of each actor. isin(targets) is True when a score is any of the listed numbers - a short way of writing several == checks joined with OR. 26 innings match: 21 ducks, three 100s, one 110 and one 128. No innings was exactly 120.

### Example 2: apply with a custom function

```
name = 'Robert Downey Jr.'
print(name.split())
print(name.split()[0].upper())

first_names = heroes.apply(lambda x: x.split()[0].upper())
print(first_names.head())

avg = canteen.mean()
print(round(avg, 1))

label = lambda x: 'Busy day' if x > avg else 'Normal day'
print(label(72))
print(label(58))

day_type = canteen.apply(label)
print(day_type.head())
```

#### Output / Result

```
['Robert', 'Downey', 'Jr.']
ROBERT
movie
Iron Man                ROBERT
Iron Man 2              ROBERT
Iron Man 3              ROBERT
The Avengers            ROBERT
Avengers: Age of Ultron ROBERT
Name: lead_actor, dtype: str
70.9
Busy day
Normal day
0    Normal day
1    Normal day
2    Normal day
3     Busy day
4    Normal day
Name: Samosas_Sold, dtype: str
```

#### Explanation

Testing on one name first: `split()` breaks it into words and `[0].upper()` gives ROBERT. `apply()` runs the same logic on every row. 70.9 is the average; `label(72)` gives Busy day and `label(58)` gives Normal day. Finally `apply(label)` labels every day in the Series.

#### OUR LOCATIONS

## 16. Plotting a Series

Because a Series pairs one axis of labels with one axis of values, it plots directly with a single `.plot()` call - no separate x and y arguments needed.

### Syntax

```
s.plot(kind='line'); s.plot(kind='bar')
```

### Example 1: Canteen sales as a line chart

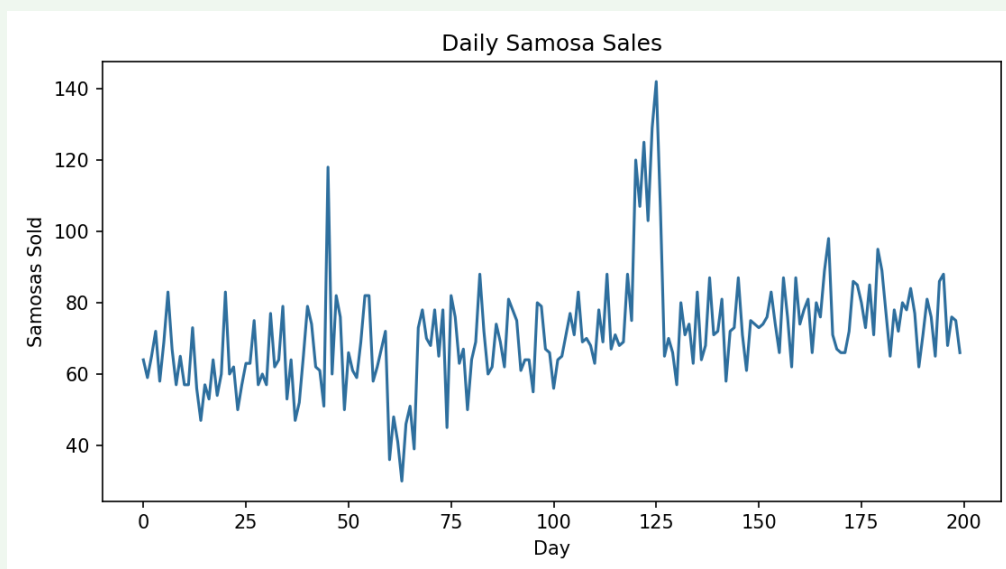
```
import matplotlib.pyplot as plt

print(canteen.size)
print(canteen.idxmin())
print(canteen.min())
print(canteen.idxmax())
print(canteen.max())

canteen.plot(color='#2E6F9E')
plt.title('Daily Samosa Sales')
plt.xlabel('Day')
plt.ylabel('Samosas Sold')
plt.show()
```

### Output / Result

```
200
63
30
125
142
```



### OUR LOCATIONS

Explanation

Before plotting we print the facts to look for: 200 days, the lowest sale on day 63 (30 samosas) and the highest on day 125 (142 samosas) - `idxmin()` and `idxmax()` return the label of the lowest and highest value. In the chart, the index becomes the x-axis and the sales become the y-axis: see the exam-week dip around days 60-66 and the festival-week spike around days 120-126.

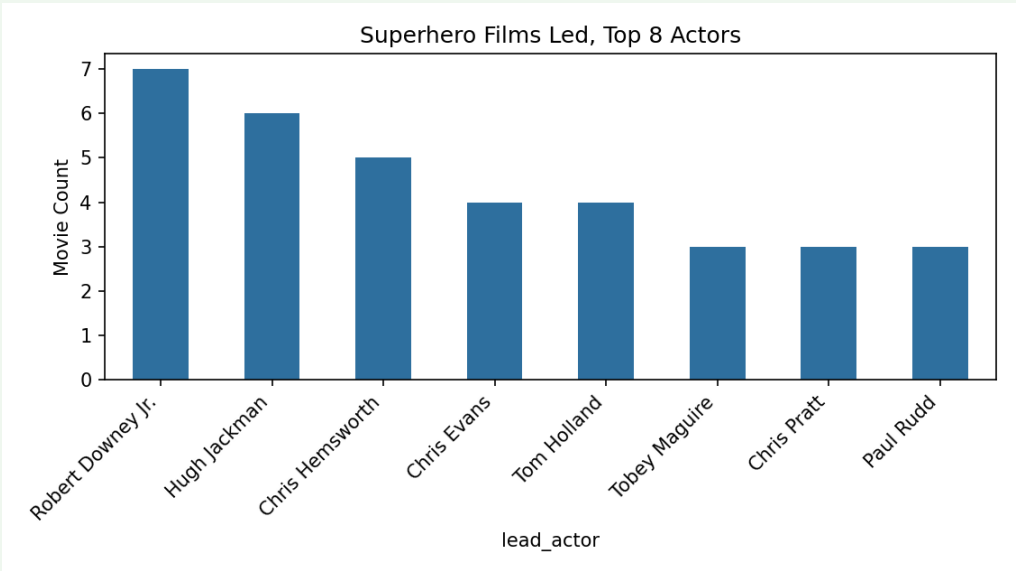
Example 2: Top lead actors as a bar chart

```
counts = heroes.value_counts()
top8 = counts.head(8)
print(top8)

top8.plot(kind='bar', color='#2E6F9E')
plt.title('Superhero Films Led, Top 8 Actors')
plt.ylabel('Movie Count')
plt.xticks(rotation=45, ha='right')
plt.show()
```

Output / Result

```
lead_actor
Robert Downey Jr.      7
Hugh Jackman           6
Chris Hemsworth        5
Chris Evans            4
Tom Holland            4
Tobey Maguire          3
Chris Pratt            3
Paul Rudd              3
Name: count, dtype: int64
```



**Explanation**

We first prepare the data - `value_counts()` then `head(8)` - and print it, so the chart has no surprises. In the bar chart the index (actor names) becomes the bar labels and the counts become the bar heights. `rotation=45` tilts the long names so they don't overlap.